

1. API Design & Data Choice: Guide Questions: What custom domain/topic did you choose, and why? What dataset dictionary keys did you define in your index.py, and why are those attributes important for describing your items?
 - For the topic, I chose Dead by Daylight, I added all characters including both survivors and killers. I used string and int for their attributes, I added their origin, realm, dlc, perks, powers, code names, year released, and their short description.
2. Guide Questions: How did you modify your routes in index.py? How does your search function filter through your custom text fields?
 - I modified the @app.get(/, /characters, and /characters/search to accept FastAPI response. Each route executes set_no_cache_headers before returning data where it remains updated regardless of serverless caching on vercel.

```
@app.get("/")
def home(response: Response):
    set_no_cache_headers(response)
    return {
        "message": "Welcome to the Dead by Daylight Character Directory API!",
        "total_characters": len(characters),
        "survivors_count": len([c for c in characters if c["role"] == "Survivor"]),
        "killers_count": len([c for c in characters if c["role"] == "Killer"]),
        "endpoints": [
            "/characters",
            "/characters/{character_id}",
            "/characters/search"
        ]
    }
```

This root get route sends welcome message, live counts for my characters, and list of all available endpoints.

CS Elective 1: Cloud Computing Development

```
# GET ALL CHARACTERS (WITH ROLE FILTER, PAGINATION, AND SORTING)
@app.get("/characters")
def get_characters(
    response: Response,
    role: str = Query(None, description="Filter by 'Survivor' or 'Killer'"),
    sort_by: str = Query("name", description="Sort field: 'name', 'year', 'difficulty', 'character_code'"),
    order: str = Query("asc", description="'asc' or 'desc'"),
    limit: int = Query(10, ge=1, le=100, description="Items per page (default: 10)"),
    offset: int = Query(0, ge=0, description="Page starting offset")
):
    set_no_cache_headers(response)
    results = characters

    # Filter by role
    if role:
        results = [c for c in results if c["role"].lower() == role.lower()]
```

This code accepts optional parameter for filtering role and paginating the results. It also ensure the data is fresh as I encounter an error recently where my code doesn't update.

```
# SEARCH CHARACTERS BY ANY KEYWORD
@app.get("/characters/search")
def search_characters(
    response: Response,
    q: str = Query(..., min_length=1),
    limit: int = Query(10, ge=1, le=100),
    offset: int = Query(0, ge=0)
):
    set_no_cache_headers(response)
    q = q.lower()
    matched = []
    for c in characters:
        searchable_text = (
            f"{c['character_code']} "
            f"{c['name']} "
            f"{c['role']} "
            f"{c['gender']} "
            f"{c['origin']} "
            f"{c['realm']} "
            f"{c['dlc']} "
            f"{c['year']} "
            f"{c['perk_1']} "
            f"{c['perk_2']} "
            f"{c['perk_3']} "
            f"{c['difficulty']} "
            f"{c.get('power', '')}"
        ).lower()

        if q in searchable_text:
            matched.append(c)

    paginated_results = matched[offset : offset + limit]
    return {
        "query": q,
        "total": len(matched),
        "limit": limit,
        "offset": offset,
        "results": paginated_results
    }
```

This defines the characters/search endpoint, it makes a single lowercase string combining 13 fields for all characters, matching a search returns a paginated result.

CS Elective 1: Cloud Computing Development

```
Pretty-print [ ]  
{"message": "Welcome to the Dead by Daylight Character Directory API!", "total_characters": 96, "survivors_count": 54, "killers_count": 42, "endpoints": [{"characters", "/characters/{character_id}", "/characters/search"}]}
```

-
3. Guide Questions: What directory structure did you set up so Vercel could recognize your code as a serverless function? What happened when you pushed your customized code updates to GitHub?
- For the backend deployment, I organized the project by following the instruction given at the lms by putting `index.py` file inside an `api/` folder right at the root of the repository to host the `app = FastAPI()` instance. By following this standard directory layout, Vercel automatically recognized the setup and built the backend as a Python serverless function, exposing it under the `/api` route while seamlessly serving the `index.html` and `app.js` frontend files alongside it.